Classification: Internal

JENKINS PIPELINE OPTIMIZATION & SONARQUBE INTEGRATION

TABLE OF CONTENTS

1.Objective 1

2.Scope 1

3.Governance Controls to be Implemented 1

4.Implementation Strategy 1

Phase 1 – Assessment (DONE) 1

Phase 2 – Pilot Implementation 1

Phase 3 – Standard Pipeline Model 2

Phase 4 – Gradual Rollout 2

5. PIPELINE MODEL 2

6. Naming Convention Standard 2

7. Sonarqube project name 2

8. Quality Gate Policy 3

9. Project Timeline (Proposed) 3

10. Dependencies 3

Classification: Internal

1.OBJECTIVE

This project aims to standardize CI/CD governance, security controls, and quality enforcement across all Java and React applications. The initiative will implement SonarQube quality gates, security scanning, naming conventions, and deployment promotion controls aligned with ITQMS requirements.

2.SCOPE

- Technologies in Scope:
  - Java
  - React JS
- Environments:

  DEV → QC → UAT → PREPROD → PROD
- name is mapped to EPIC name.

3.GOVERNANCE CONTROLS TO BE IMPLEMENTED

- A new pipeline for Secret detection + Sonarqube scanner + build in one pipeline.
- CI/CD naming convention validation.
- SonarQube integration and Quality Gate enforcement.
- Code coverage validation based on automated test execution.
- Security scanning & secret detection.
- Full traceability from Bitbucket → Jenkins → Deployment.

4.IMPLEMENTATION STRATEGY

The implementation will follow a phased approach to ensure smooth adoption and minimize disruption to development teams.

PHASE 1 – ASSESSMENT (DONE)

- Create Inventory all Java and React repositories and identify existing Jenkins pipelines.
- Evaluate SonarQube integration status for current Jenkins Pipelines.

PHASE 2 – PILOT IMPLEMENTATION

Instead of updating all applications at once, the rollout will be performed gradually by technology and application group. And this allows to validate the pipeline configuration, monitor build stability, and address issues early before rollout to all other applications.

- Will start with one Java application and one React application.
- Implement standardized Jenkins pipeline.
- Confirm security scanning integration.
- Integrate SonarQube analysis within build pipeline.

- Configure Quality Gate with required Options to validate enforcement.

PHASE 3 – STANDARD PIPELINE MODEL

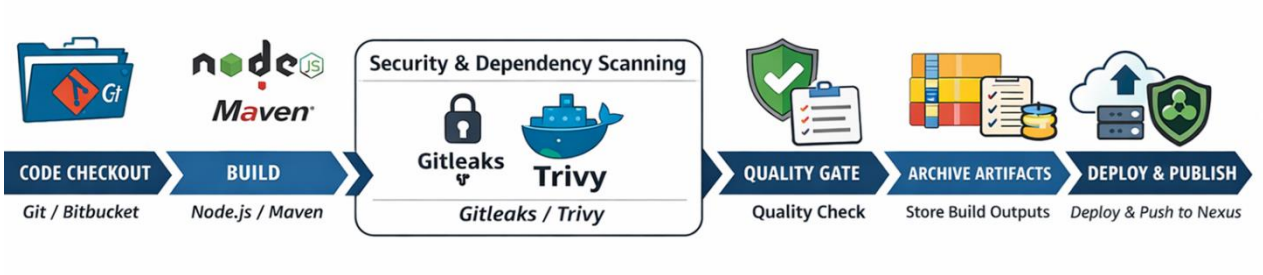
- Implement shared Jenkins library for standardized pipelines
- Ensure build, test, scan, and quality gate validation are included
- Apply naming validation and deployment gates.

PHASE 4 – GRADUAL ROLLOUT

- Roll out standardized pipelines to remaining Java and React applications

5. PIPELINE MODEL

Standard pipeline flow:



6. NAMING CONVENTION STANDARD

Currently Release name is set with related EPIC Name so in order to improve consistency w ewe will use release name with compination of EPIC Name & Release Name.

- **Release Naming Format**

  <EPIC NAME> ⇔ R<ReleaseNumber>-<LOB>-v<Major.Minor.Patch>-<Environment>

  Example

  EPIC8033 ⇔ R25-DXP-v1.0.0-DEV

7. SOANRQUBE PROJECT NAME

We can set Naming standard for each project

- **Project Naming Format**

  <LOB>-<Jenkins Job Name>

  Example

  DXP-BFF-AAMAL-ESERVICES

- **Versioning**

  R<ReleaseNumber>-<LOB>-v<Major.Minor.Patch>-<Environment>

  Example

8. QUALITY GATE POLICY

Quality Gate rules include:

- No Blocker or Critical vulnerabilities.
- No Blocker or Critical bugs.
- Code coverage on new code ≥ 40% (initial threshold).
- Code duplication < 5%.
- No new critical code smells.
- Security hotspots must be reviewed.

Note:

First, We can start with the minimal configurations for Quality gate to ensure that it works fine then align with the corresponding team to solve discovered issues.

9. PROJECT TIMELINE (PROPOSED)

Phase	Activity	Estimated Duration	Status
Phase 1	Assessment & inventory	1 week	Done
Phase 2	Standard Pipeline Development (Pilot Apps)	1 Week	
Phase 3	Java applications rollout	3-4 weeks	
Phase 4	React applications rollout	3-4 weeks	

10. DEPENDENCIES

- Development teams support for test coverage and vulnerability fixes.